

INTEGRATING ERC-20 ALLOWANCE EDGES INTO PAGERANK FOR ENHANCED ON-CHAIN WALLET REPUTATION SCORING

CHAPTER 3: Research Methodology

by

TAEHONG KWON

Student number: 28576810

submitted in accordance with the requirements for the degree of

DOCTOR OF PHILOSOPHY

in the subject

COMPUTER SCIENCE

at the

UNIVERSITY OF SOUTH AFRICA

SUPERVISOR: Prof. Ernest Mnkandla

CO-SUPERVISOR: Prof. Donatien Koulla Moulla

CO-SUPERVISOR: Dr. David Sena Attipoe

July 2026

3 Research Methodology

Chapter 2 closed on a gap: no reviewed method ranks wallets on allowance edges, the transfer-graph baseline checks itself against proxies taken from its own input, and efficiency claims are rarely tied to edge counts under a fixed protocol. This chapter compares EndorseRank and Adaptive Weighted PageRank (AWP) as social reputation scores on Arbitrum One, with a measurable procedure for each of those gaps. The primary comparison is EndorseRank on the ERC-20 allowance graph against AWP on the time-decayed transfer graph ^[3], both solved with classical PageRank ^[15]. The methodological stance follows construct (domain) alignment: reputation scores are continuous rankings, and checks use observable wallet-level statistics rather than binary default labels ^[22]. In measurement terms, each ranking method is an indicator of an unobserved reputation construct. Contemporaneous transfer and allowance statistics check graph coherence. A temporal holdout of future new owner–spender approvals is the out-of-window check. Alignment statistics ask whether the ordering induced by a method co-varies with the ordering induced by each check on the same wallets. This approach is appropriate because the scientific claim is comparative (EndorseRank versus AWP) rather than a prediction of trading profit or credit default.

All procedures operate on publicly available blockchain logs within a fixed observation window, with deterministic seeds and an open evaluation pipeline so that reported tables can be regenerated from cached parquet.

Four principles govern every operational choice.

- Same-seed comparability: EndorseRank and AWP are always scored on identical wallet sets so that differences cannot be attributed to sampling inconsistency.
- Pre-registration by configuration: observation window, topic hashes, damping, decay parameters, bootstrap resamples and seed, and benchmark seeds are fixed in configuration before interpreting results.

- Construction and timing: Graph construction is performed only once per cohort. Computational benchmarks only time the PageRank solve.
- Scope: Claims C1–C4 and hypotheses H1–H3 pertain to EndorseRank and AWP, while claim C5 pertains to the coupled operator and its ablation. RQ4 and RQ5 utilize the spender holdout cohort.

3.1 Notation

Table 3.1 below lists the notation employed in this chapter and in Chapters 4 and 5 (for a condensed list see Table 2). All vectors are shown in boldface and all sets are written in calligraphic font. Wallet addresses are used as opaque identifiers and are not mapped to any external identities.

Table 3.1: Notation for graphs, scores, proxies, and parameters.

Symbol	Meaning
$\mathcal{W}$	Matched wallet cohort ($ \mathcal{W} = n = 5,521$)
$\mathcal{W}_{\text{pool}}$	Expanded wallet pool ($ \mathcal{W}_{\text{pool}} = N = 99,080$)
$G_{\text{approve}} = (V_A, E_A, w_A)$	EndorseRank allowance graph
$G_{\text{transfer}} = (V_T, E_T, w_T)$	AWP transfer graph
$w_A(u, v)$	Latest allowance weight, owner $u \rightarrow$ spender v
$w_T(u, v)$	Time-decayed transfer weight, sender $u \rightarrow$ receiver v
$\mathbf{s}_M$	Reputation score vector for method M
$\mathbf{p}_P$	Proxy value vector for proxy P
d	PageRank damping factor (default 0.85)
τ_{tol}	PageRank convergence tolerance (10^{-8})
I_{max}	Maximum PageRank iterations (300)
$\sigma(\Delta t)$	Logistic time-decay on transfer age
k, t_0	Decay steepness (0.01) and midpoint (180 days)
λ	Allowance-layer weight of the coupled operator C-PR ($\lambda = 0.5$ primary; 0.25, 0.75 sensitivity)
π	Teleportation distribution (uniform; equal to $\mathbf{s}_{\text{ER}}$ for S-PR)
T_1, T_2	Holdout freeze date (2026-02-28 UTC) and outcome window (2026-03-01 to 2026-05-31 UTC)
Δt	Age of an event in days before observation end
ρ	Spearman’s rank correlation
τ	Kendall’s rank correlation
T_{obs}	Observation end timestamp (2026-05-31 UTC)

Unless otherwise noted, PageRank is the weighted variant of Page et al.^[15], defined at iteration t as

$$\mathbf{r}^{(t+1)} = (1 - d) \boldsymbol{\pi} + d P^\top \mathbf{r}^{(t)} + d (\mathbf{r}^{(t)\top} \mathbf{1}_{\text{dang}}) \boldsymbol{\pi}, \quad (3.1)$$

where P is the row-stochastic transition matrix generated from positive edge weights, $\boldsymbol{\pi}$ is the uniform teleportation distribution across nodes, and $\mathbf{1}_{\text{dang}}$ denotes the set of dangling nodes (i.e., those with zero out-strength). Specifically, if $w(u, v)$ is the weight of the directed edge from u to v , and $s(u) = \sum_v w(u, v)$ is the out-strength, then

$$P_{u,v} = \begin{cases} w(u, v)/s(u) & \text{if } s(u) > 0, \\ 0 & \text{otherwise.} \end{cases} \quad (3.2)$$

Dangling mass is redistributed uniformly via the teleportation term, following standard conventions for sparse web and transaction graphs. The algorithm terminates when $\|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}\|_1 < \tau_{\text{tol}}$ or after I_{max} iterations. Scores are normalized such that they sum to one across all reachable nodes. Ranks for alignment are computed by sorting the scores in descending order (rank 1 indicates the highest score), using wallet address as a secondary, deterministic ordering criterion in case of ties. The damping factor d interpolates between purely graph-based propagation (as $d \rightarrow 1$) and uniform scores (as $d \rightarrow 0$). The default $d = 0.85$ reflects the classical web setting and the AWP baseline^[3]. In the robustness sweeps we vary d while keeping edge weights constant (Section 3.12). We choose $\tau_{\text{tol}} = 10^{-8}$ and $I_{\text{max}} = 300$ to be more restrictive than in typical interactive analytics contexts to ensure that runtimes measure graph structural properties rather than early termination of iteration.

3.2 Data sources and sampling

This subsection defines the empirical space: the chain, time interval, public tables, events, and wallets that comprise the main sample. Any subsequent construction of the graph or proxy does not introduce events beyond this space.

- **Primary chain:** The entire analysis draws from events occurring on Arbitrum One (chain ID 42161), an Ethereum Layer-2 rollup with high DeFi usage and publicly available transaction logs ^[1]. Focusing on one chain eliminates the problem of cross-chain aliases (a single user operating different addresses on multiple chains would generate distinct node entries) and ensures uniform gas fees and network latencies for all wallets in the sample. The sample chain is the only empirical data source in our comparison.
- **Observation window:** Our time range is 2025-12-01 00:00:00 UTC to 2026-05-31 23:59:59 UTC (six months). In the SQL queries, the final exclusive timestamp is 2026-06-01 00:00:00 UTC, such that the inclusive upper bound is 2026-05-31. This period is long enough to fit within the BigQuery data limits for a month-by-month extraction but short enough that the snapshot dates for max allowance match the date ranges of the transaction history. AWP’s time-decay factor is calculated using an anchor at the final moment of the observation period, $T_{\text{obs}} = 2026-05-31\ 23:59:59\ \text{UTC}$, so an event on the last day of the window has $\Delta t = 0$ and an event on the first day has $\Delta t \approx 181$ days.
- **BigQuery source:** Logs are pulled from the public BigQuery table

`bigquery-public-data.google_blockchain_arbitrum_one_us.logs`.

Each record is a single event emitted by a contract, containing information about the block timestamp, address, event topics, and event data. Queries select records based on block timestamp, contract address (if applicable), and event topic hashes. Using the public dataset means there is no need for a dedicated archival node and it is fully verifiable: anyone with a Google Cloud account can run the exact same query on the same table. The queries used, along with their estimated costs and other metadata, are provided in Appendix 8.1.

- **Reputation events:** EndorseRank relies on ERC-20 Approval logs ^[18], which includes approvals made through EIP-2612 permit transactions ^[19]. AWP relies on ERC-20 Transfer logs. In both cases, we restrict the events to include only those where the owner or spender (for approval events) or the sender or receiver (for transfer events) is in the sample of wallets.

Filtering at the query stage minimizes the data volume extracted from the chain compared to pulling all chain events, and isolates the subgraph relevant to our cohort’s interactions. We run the extraction in six monthly batches because (a) there are per-query byte limits, and (b) in case one month of extraction fails, we need a way to resume from where we left off.

- **Matched cohort:** The main sample of wallets to which we apply alignment are wallets with non-zero connectivity in the reputation subgraph defined by approval or transfer edges within the observation window. We preprocess this extracted cohort after the BigQuery extraction phase, yielding a matched wallet cohort of size $n = 5,521$. We evaluate EndorseRank and AWP on the same seed set so that any differences in alignment and runtime can be attributed to the ranking logic and the structure of the graph, and not to the effect of sampling differences.
- **Exclusions:** We do not process information about mempool transactions, off-chain order books, centralized exchange user accounts, nor multi-chain bridges. We do not attempt to resolve entities that correspond to multiple addresses controlled by a single operator. Our public log table is the ground truth and our decoding happens in the open pipeline.

3.3 Two-phase collection strategy

We split our data collection and evaluation into two phases. The first phase collects the data. In the second phase we conduct all evaluation locally. This separation allows us to conduct alignment tables, benchmarking and robustness experiments many times without incurring cloud costs – a necessity for other researchers to replicate our results.

- **Phase 1 (reputation streams):** Extract all ERC-20 Approval and Transfer events that were emitted by or affect wallets in our matched cohort ($n = 5,521$ wallets). For each month, we use a separate query, so if one query fails, it does not impact the results of the previous ones. Each query also gets its own dry-run estimate, ensuring we know how much we will be charged ahead of time. We preprocess the approvals to keep only the most recent allowance for each owner–spender–token triple. If we see several approvals for the same triple in the

window, only the one with the latest timestamp is kept. If we observe an allowance of 0, the edge is dropped. We preprocess the transfers into tables with decimals adjusted so that values are scaled according to the standard representation. The resulting tables contain sender, recipient, value, token and block timestamp. They serve as the sole inputs to both EndorseRank and AWP graph construction.

- Phase 2 (local evaluation): We use the preprocessed parquet files from Phase 1 to compute the following quantities, all locally: reputation ranks, transfer and allowance checks, alignment metrics (Spearman correlation, Kendall τ), computational benchmarks (time and memory consumption) and scaling/robustness extensions. For the active wallets that comprise our expanded pool, we simply add addresses in the reputation subgraphs used in our expanded pool (or supplemental parquet files). For robustness experiments, we do not need to conduct another large chain scan in order to find these wallets. Once we have the Phase 1 extracts and the supplemental active-wallet parquet files, no additional BigQuery scans are necessary.
- Cost guardrails: Before starting any remote query, we run a dry-run of the query and record the estimated number of bytes processed. If the estimated bytes processed exceed our per-query budget of 150 GiB,¹ we abort the query. The operator must confirm via an extra flag if they would like us to proceed with scanning data beyond our budget. We record information such as the number of wallets scanned, the number of rows returned, the total number of bytes processed, and the timestamp of the query. These records are available as part of our public acquisition log (see Appendix 8.1). This ensures that our study makes responsible use of public cloud resources and that we can audit the cost of the acquisition phase of this study.

3.4 Pipeline architecture

The full pipeline from public logs to research tables is visualized in Figure 3.1. All remote steps produce immutable parquet files. Each local step is deterministic given its inputs and the versioned

¹The per-query byte limit is a constraint enforced by the pipeline of this study, and not a limitation imposed by BigQuery.

configuration file.

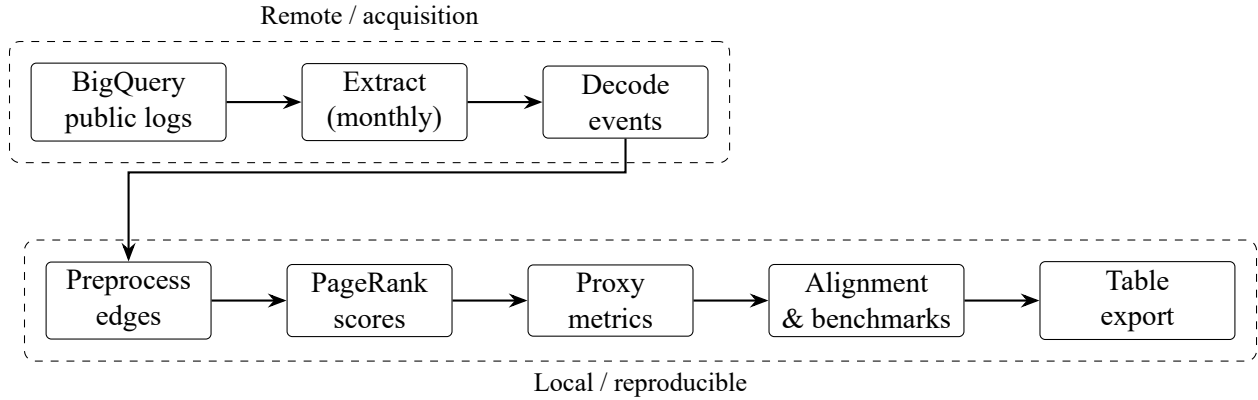


Figure 3.1: Pipeline overview: Arbitrum public logs are parsed and decoded, edge records are prepared, EndorseRank and AWP scores are calculated, alignment and benchmark calculations are performed, and output tables are written.

The Phase 1 scripts create monthly approval and transfer event shards as well as the aggregated latest-allowance and transfer-event tables. The Phase 2 scripts consume these tables, construct a graph, calculate PageRank scores, and generate the result tables presented in Chapter 4. The pipeline can be executed entirely offline using a synthetic fixture dataset without access to the cloud.

3.5 Event decoding

Logs emitted by a smart contract consist of topics and a data payload, but do not contain field names. The decoding step creates a typed record for each log row which is then consumed by graph construction and check generation. If this step were performed incorrectly, it would silently poison both reputation graphs. This section details the topic hashes and field mappings used in the decoding step.

- **Log anatomy:** A smart contract log consists of the address of the emitting contract, a sequence of topics, and an ABI-encoded data blob. The first topic is always the keccak-256 hash of the event signature string, which uniquely identifies the type of event among standard specifica-

tions. Indexed parameters are stored as 32-byte words in subsequent topics (with addresses left-padded), while non-indexed parameters are stored in ABI order in the data field^[18]. These fields are exposed in BigQuery as table columns. Topic filtering is done with SQL, while the decoding of fields with numeric values is handled by the Python script.

- ERC-20 Approval: The topic hash is

`0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925,`

which is the keccak-256 hash of the string `Approval(address,address,uint256)`. Topics 1 and 2 contain the owner and spender addresses (left-padded), while the allowance value is decoded from the data field and adjusted by the token's decimal value. Large allowance values (equivalent to `type(uint256).max`) are also preserved after decimal adjustment. Unlimited allowances are included because they represent strong authorization on-chain. The ERC-2612 permit standard does not emit a new event type; the permit log contains the same Approval event, which means that the EndorseRank scores will take into account the resulting allowance regardless of whether the authorization was granted via an on-chain `approve()` call or an off-chain signature^[19].

- ERC-20 Transfer: The topic hash is

`0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef,`

which is the keccak-256 hash of the string `Transfer(address,address,uint256)`. The sender and recipient are indexed, while the transfer value is decimal-adjusted. Transfer logs which involve minting and burning tokens using the zero address are preserved if they involve any of the cohort wallets, but such events are infrequent compared to the volume of wallet-to-wallet transfers in the filtered extract. Self-transfers are included in the events table, but do not count toward Sybil-stability inbound-counterparty computations (Section 3.9), since a transfer from a wallet to itself doesn't indicate any kind of counterparty.

- Address normalization and units: all addresses are lower-cased prior to the join, to ensure that checksummed and non-checksummed hex representations collide appropriately. Token

amounts are represented as floating point decimals adjusted for the number of decimal places of each token, to serve as the value for computing a weighted amount; however, when determining rankings, we use relative amounts within a cohort rather than converting them to absolute USD values to compute allowance and transfer edges.

3.6 Graph construction

EndorseRank and AWP both use the PageRank solver in Equation (3.1) but define their edges differently. Both approaches create a directed weighted graph using the wallet addresses as nodes, consider only the subgraph involving the seed wallet set (edges where at least one of the two endpoints is in the seed wallet set, including 1-hop neighbor addresses), and return a score for every address in the seed wallet set (scores of 0 are assigned before ranking for any addresses not in the subgraph). Subgraph restriction is necessary both for performance reasons, and for clarity of interpretation: the score for a wallet address in the seed wallet set may accrue score mass from addresses outside of the seed wallet set which approve or transfer to it, but we report scores only for the addresses in the seed wallet set, i.e. $\mathcal{W}$.

The key difference is between endorsements and transfers. An allowance is a deliberate, and revocable, authorization for someone else to spend tokens on your behalf; whereas a transfer is a completed transfer of token value, which could be part of payments, routing, market making or even wash trading, none of which require any sort of endorsement commitment ^{[4],[18]}. Thus, EndorseRank can be thought of as treating the latest snapshots of allowance data as a network of endorsements, similar to a citations network in Page et al.^[15]. By contrast, AWP treats each instance of a transfer as a citation, weighted according to how long ago that transfer occurred, such that more recent transfers have a greater impact than older ones.

3.6.1 EndorseRank allowance graph

Let $\mathcal{A}$ be the collection of latest allowances after preprocessing: for each (token, owner–spender) pair, only the chronologically last Approval event within the observation window is considered (and a zero allowance removes the edge). Across multiple tokens, the allowance weights are summed:

$$w_A(u, v) = \sum_{\text{token}} a^*(u, v, \text{token}), \quad (3.3)$$

where a^* represents the latest decimal-adjusted allowance. The directed edge $u \rightarrow v$ signifies owner u is endorsing spender v . The damping factor is $d = 0.85$. We do not apply any time-based decay, because overrides based on recency of an approval replace time-based weights for identical owner–spender pairs ^[18].

Algorithm 1 describes the process to construct the graph and run PageRank.

Algorithm 1 EndorseRank graph construction and scoring

Require: Latest allowances $\mathcal{A}$, seed wallets $\mathcal{W}$, damping d , tolerance τ_{tol} , max iterations I_{max}

Ensure: Scores $\mathbf{s}_{\text{ER}}$ on $\mathcal{W}$

```

1:  $E \leftarrow \emptyset$ 
2: for each  $(u, v)$  in unique owner–spender pairs in  $\mathcal{A}$  do
3:    $w \leftarrow \sum_{\text{token}} a^*(u, v, \text{token})$ 
4:   if  $w > 0$  then
5:     add edge  $(u \rightarrow v, w)$  to  $E$ 
6:   end if
7: end for
8:  $E \leftarrow$  edges in  $E$  incident to at least one wallet in  $\mathcal{W}$ 
9:  $\mathbf{s} \leftarrow \text{WeightedPageRank}(E, d, \tau_{\text{tol}}, I_{\text{max}})$ 
10: return  $\mathbf{s}_{\text{ER}}[w] \leftarrow \mathbf{s}[w]$  for  $w \in \mathcal{W}$  (else 0)

```

3.6.2 AWP transfer graph

AWP adopts the approach described by IEEE RIVF 2023^[3], in which each transfer of token value v at age Δt days prior to the end of the observation window T_{obs} contributes a decayed weight

$$\sigma(\Delta t) = \frac{1}{1 + \exp(k(\Delta t - t_0))}, \quad k = 0.01, \quad t_0 = 180 \text{ days.} \quad (3.4)$$

Edge weights aggregate over transfers:

$$w_T(u, v) = \sum_{e: u \rightarrow v} v_e \sigma(\Delta t_e). \quad (3.5)$$

Damping is again $d = 0.85$. Figure 3.2 plots $\sigma(\Delta t)$ over the relevant age range: recent transfers retain nearly full weight, while transfers older than the midpoint t_0 are smoothly down-weighted.

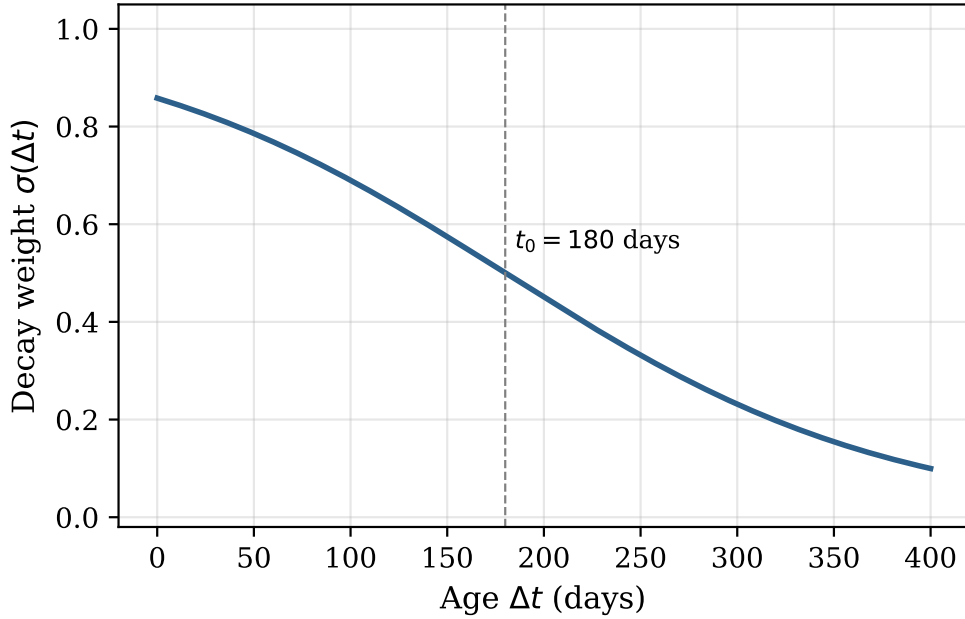


Figure 3.2: Logistic time-decay $\sigma(\Delta t)$ used by AWP ($k = 0.01$, $t_0 = 180$ days). Horizontal axis is event age in days before 2026-05-31 UTC; vertical axis is the multiplicative weight applied to transfer value.

Algorithm 2 formalizes construction and scoring.

Algorithm 2 AWP graph construction and scoring

Require: Transfers $\mathcal{T}$, seed wallets $\mathcal{W}$, observation end T_{obs} , parameters $k, t_0, d, \tau_{\text{tol}}, I_{\text{max}}$

Ensure: Scores $\mathbf{s}_{\text{AWP}}$ on $\mathcal{W}$

```
1:  $E \leftarrow \emptyset$ 
2: for each transfer  $e = (u, v, v_e, t_e)$  in  $\mathcal{T}$  do
3:    $\Delta t \leftarrow (T_{\text{obs}} - t_e)$  in days
4:    $w_e \leftarrow v_e \cdot \sigma(\Delta t)$  with  $\sigma$  as in Equation (3.4)
5:   accumulate  $w_e$  on edge  $(u \rightarrow v)$ 
6: end for
7: remove non-positive edges; retain edges incident to  $\mathcal{W}$ 
8:  $\mathbf{s} \leftarrow \text{WeightedPageRank}(E, d, \tau_{\text{tol}}, I_{\text{max}})$ 
9: return  $\mathbf{s}_{\text{AWP}}[w] \leftarrow \mathbf{s}[w]$  for  $w \in \mathcal{W}$  (else 0)
```

Because G_{approve} records durable authorizations and G_{transfer} records historical value flows, the two graphs differ in density and temporal sensitivity. Approvals are relatively rare, high-intent events; transfers are frequent and often mechanical. On the matched cohort, the approval subgraph is substantially sparser than the transfer subgraph (Chapter 4), which is the structural basis for the computational-efficiency hypothesis (H3): per-iteration PageRank cost scales with $|E|$, so fewer edges imply faster solves and lower memory for the adjacency structures in Equation (3.2).

Both methods share the same implementation details. Edge lists are stored as triples (from node, to node, weight) in memory. Duplicate edges are aggregated by summing weights before the solve. Non-positive weights are dropped. The solver builds a sparse compressed-row transition matrix, iterates Equation (3.1), and returns a dictionary from node identifier to score. Cohort scores are aligned to $\mathcal{W}$ with missing entries set to zero, then converted to dense ranks for correlation.

3.6.3 Coupled operator over both layers (C-PR) and the seeded ablation (S-PR)

EndorseRank and AWP each walk on one edge type. RQ5 asks whether a single walk that uses both edge types ranks wallets better than either. The allowance graph and the transfer graph are treated as two layers over the union node set $V = V_A \cup V_T$, with layer transition matrices P_A and P_T defined by Equation (3.2) on w_A and w_T respectively. The coupled transition row of wallet u

is a convex mixture of its two layer rows, renormalized over the layers in which u has out-edges:

$$P_{u,\cdot}^{(\lambda)} = \alpha_A(u) P_{A,u,\cdot} + \alpha_T(u) P_{T,u,\cdot}, \quad \alpha_A(u) = \frac{\lambda \mathbb{I}[s_A(u) > 0]}{\lambda \mathbb{I}[s_A(u) > 0] + (1 - \lambda) \mathbb{I}[s_T(u) > 0]}, \quad \alpha_T(u) = 1 - \alpha_A(u) \quad (3.6)$$

where $s_A(u)$ and $s_T(u)$ are the out-strengths of u in each layer. A wallet with out-edges in one layer only follows that layer with weight one, so no step is lost to a layer the wallet does not use; a wallet with no out-edges in either layer is dangling and is treated as in Equation (3.1). Teleportation is uniform and $d = 0.85$. The score vector $\mathbf{s}_{\text{CPR}}$ is the stationary vector of $P^{(\lambda)}$ under the same iteration, tolerance and cap as the single-layer methods.

The operator nests both compared methods. Setting $\lambda = 1$ makes every mixed row equal to its allowance row and $\lambda = 0$ to its transfer row; on a shared node set the two end points are therefore EndorseRank and AWP exactly, and the unit tests of the pipeline check this identity on a fixture. On the empirical graphs the two layers differ in their one-hop neighbors, so the end points reproduce the single-layer rankings on the common wallets rather than the full score vectors. C-PR is not a third edge swap but a one-parameter family whose ends are the two methods under comparison, and RQ5 asks whether an interior point beats either end. The primary value $\lambda = 0.5$ and the sensitivity values $\lambda \in \{0.25, 0.75\}$ were fixed in the versioned configuration on 14 September 2026, before any coupled score was computed.

The ablation S-PR keeps the transfer walk and replaces the uniform teleportation vector π in Equation (3.1) with the normalized EndorseRank score, $\pi = \mathbf{s}_{\text{ER}} / \|\mathbf{s}_{\text{ER}}\|_1$ on V_T (wallets without an EndorseRank score receive zero restart mass). Dangling mass is redistributed with the same vector. This is the construction of TrustRank^[17] with allowance-derived seeds in place of hand-labeled ones: allowances decide where the walk restarts, transfers decide how it spreads. Comparing S-PR with C-PR separates the contribution of the allowance edge as a propagation medium from its contribution as a seed.

The cost of the coupled operator follows from its edge set. C-PR walks on $E_A \cup E_T$, so its per-iteration cost is that of AWP plus that of EndorseRank, and its runtime is expected to sit at or

above AWP's; the speed result of Claim C1 therefore belongs to EndorseRank alone. S-PR uses 1 EndorseRank solve and 1 transfer solve. The two operators are timed using the benchmark protocol from Section 3.11.

3.7 Temporal holdout protocol

The holdout scores are computed from all events with timestamp at or before $T_1 = \text{February 28, 2026, 23:59:59 UTC}$. The labels use events between $T_2^{\text{start}} = \text{March 1, 2026}$ and $T_2^{\text{end}} = \text{May 31, 2026}$. The cohort is the set of t_1 inbound spenders ($n = 1,335$), the wallets holding at least one positive allowance in the latest-positive snapshot at T_1 . All scores in the holdout table are computed from t_1 events only: EndorseRank, AWP, C-PR at the three values of λ , S-PR, and two raw-degree baselines.

Two labels are counted on each wallet, one per construct, so that each single-layer method is tested on its own construct in the future and the hybrids are tested on both:

1. New approval pairs: the number of owner–spender pairs observed in T_2 that were not present in the latest-positive t_1 snapshot and are attributed to the spender. This is the endorsement construct.
2. New transfer senders: the number of distinct addresses that send a transfer to the wallet in T_2 and did not do so at or before T_1 (excluding self-transfers). This is the flow construct and it matches the first label on the transfer side.

The two baselines are the in-approve degree (number of distinct owners with positive allowance) and the transfer in-degree (number of distinct senders) of each wallet at T_1 . They are the local statistics smoothed by each single-layer PageRank, and their rows allow us to read off the increment of each PageRank over its own raw degree directly: RQ4 is answered by this increment, not by the PageRank coefficient itself.

Uncertainty is computed as described in Section 3.10, a paired wallet bootstrap with $B = 400$ resamples and one shared index matrix, so that every difference between two coefficients on the

same label is a paired difference. Eight contrasts were fixed prior to the run: EndorseRank minus in-approve degree on new approval pairs; AWP minus transfer in-degree on new transfer senders; EndorseRank minus AWP on new approval pairs and AWP minus EndorseRank on new transfer senders; C-PR minus EndorseRank on new approval pairs and C-PR minus AWP on new transfer senders; the same two contrasts for S-PR.

The pre-registered rule for RQ5 has a primary and a secondary part. Primary (holdout): C-PR at $\lambda = 0.5$ should be no worse than EndorseRank on new approval pairs (interval of $\Delta\tau = \tau_{\text{CPR}} - \tau_{\text{ER}}$ includes zero or lies above it), and should be better than AWP on new transfer senders (interval of $\tau_{\text{CPR}} - \tau_{\text{AWP}}$ lies above zero). Secondary (same window, matched cohort): C-PR falls in the interval of the better single-layer method on the transfer and allowance families and exceeds both single-layer methods on the Sybil-stability family by intervals that exclude zero. When the primary rule fails, RQ5 is answered negatively and the result is reported along with its intervals; λ is not tuned once labels are observed.

Other t_2 quantities (revoke rate, an approval-then-outbound drain heuristic, Aave borrower liquidations) were computed in the pipeline. They are not reported as supporting evidence: revoke and drain had the wrong sign relative to an endorsement reading, and Aave events on this spender set were empty or the wrong construct.

3.8 Construct-check design

The primary evaluation compares EndorseRank and AWP only. For every method M and every check P , the pipeline computes scores $\mathbf{s}_M$ and check values $\mathbf{p}_P$, and reports Spearman's ρ and Kendall's τ with 95% bootstrap intervals (Section 3.10). Checks are oriented such that higher values indicate higher expected reputation prior to correlation, consistent with the AWP validation protocol^[3].

There are two contemporaneous families that structure the same-window comparison:

1. Transfer (AWP baseline proxies^[3]): inbound transfer activity.

2. Allowance: inbound approval activity (wallet as spender).

They are graph-adjacent: they are computed from the same event classes that form G_{transfer} and G_{approve} , but they are local statistics rather than global PageRank scores. If AWP aligns strongly with transfer checks, or EndorseRank aligns strongly with allowance checks, this demonstrates within-construct coherence rather than independent predictive power. Sybil-adjusted activity summaries are provided in the robustness tables only, and do not form a headline claim.

The out-of-window check is the temporal holdout used in Section 3.7. Packin and Lev-Aretz^[2] warn that decentralized credit scores can become opaque; publishing explicit formulas (Section 3.9) and distinguishing same-window checks from the time split mitigates this opacity.

3.9 Operational definitions of proxies

Let $\mathcal{W}$ be the matched cohort. The notation below follows the check-computation module of the open evaluation pipeline.

3.9.1 Transfer family

Let $\mathcal{T}_{\text{in}}(w)$ denote inbound transfers to wallet w (from any sender). We define

$$\text{in_degree}(w) = |\{ u : \exists \text{ transfer } u \rightarrow w \}|, \quad (3.7)$$

$$\text{in_value}(w) = \sum_{e: \cdot \rightarrow w} v_e. \quad (3.8)$$

These correspond to the domain-intuition baselines defined in the AWP paper^[3]: economically important addresses tend to receive transfers from more distinct counterparties and in a greater total value. Unlike PageRank, in_degree and in_value are local to each address: they do not consider the reputations of the senders. Comparing AWP (which is global and time-decayed) against these local baselines tests whether propagation and recency weighting contribute additional ordering information above and beyond raw inflow magnitude.

3.9.2 Allowance family

Let $\mathcal{A}_{\text{in}}(w)$ denote the latest allowances in which w is the spender (owner $\rightarrow w$). We define

$$\text{in_approve_degree}(w) = |\{u : a^*(u, w, \cdot) > 0\}|, \quad (3.9)$$

$$\text{in_approve_value}(w) = \sum_{u, \text{token}} a^*(u, w, \text{token}). \quad (3.10)$$

These checks measure the endorsement the construct EndorseRank is designed to rank. High concordance of EndorseRank with this family is anticipated and interpreted as within-construct validity. Like the transfer baselines, the allowance checks are local; they count and sum incoming approvals without traversing multi-hop endorsement chains. Evidence beyond the time window is held out for the temporal holdout.

3.9.3 Sybil-adjusted stability family

Self-transfers ($u = w$) are omitted from the inbound counters. Let $N_{\text{tx}}(w)$ denote the number of inbound transfers to w from other addresses and $N_{\text{uniq}}(w)$ denote the number of unique inbound counterparties. Let $t_{\min}(w)$ and $t_{\max}(w)$ denote the earliest and latest timestamps where w appears as sender or recipient in the transfers table, and $M(w)$ denote the number of unique calendar months of activity (inbound months and full-span months, using the maximum). Define

$$\text{inbound_counterparty_ratio}(w) = \frac{N_{\text{uniq}}(w)}{\max(N_{\text{tx}}(w), 1)}, \quad (3.11)$$

$$\text{transfer_tenure_days}(w) = \frac{t_{\max}(w) - t_{\min}(w)}{86400} \quad (\text{seconds to days}), \quad (3.12)$$

$$\text{active_months}(w) = M(w). \quad (3.13)$$

A high counterparty ratio penalizes wallets that receive many transfers from few addresses (a simple Sybil/wash pattern). Tenure and active months reflect persistence of on-chain activity over the observation window. These are imperfect Sybil mitigations, as colluding adversaries can still main-

tain diverse counterparties, but they operationalize the intuition that sustained, multi-counterparty activity is more indicative of reputation than bursty self-dealing ^[2].

3.10 Statistical tests

Reputation scores and validation proxies are continuous rank variables, not binary default indicators. Consistent with the AWP validation protocol^[3] and the construct-validity approach of Cronbach and Meehl^[22], concordance is assessed via rank correlations that are invariant under monotonic re-scalings of either variable, and each coefficient is accompanied by a bootstrap confidence interval.

- Spearman’s ρ : Let R_i and S_i be the dense ranks of the score and proxy for wallet $i = 1, \dots, n$. Spearman’s coefficient is the Pearson correlation of these ranks:

$$\rho = \frac{\sum_{i=1}^n (R_i - \bar{R})(S_i - \bar{S})}{\sqrt{\sum_{i=1}^n (R_i - \bar{R})^2 \sum_{i=1}^n (S_i - \bar{S})^2}}. \quad (3.14)$$

Spearman’s ρ captures monotonic agreement: if greater reputation consistently corresponds to greater proxy values, ρ is positive regardless of nonlinearity in the raw scores.

- Kendall’s τ : Kendall’s coefficient counts concordant and discordant pairs. For all pairs $i < j$,

$$\tau = \frac{2(C - D)}{n(n - 1)}, \quad (3.15)$$

where C is the number of pairs ranked identically by both rankings and D is the number of pairs ranked oppositely (with ties resolved by the implementation’s standard tie adjustment). Kendall’s τ is easier to interpret as a probability that two randomly chosen items are in the correct order, and it is less affected by large rank shifts than Spearman in certain heavy-tailed distributions.

- Why both: Spearman captures a global monotonic association while Kendall puts more weight on the correct order of pairs. We report both because the AWP protocol ^[3] requires it, to ensure

that our conclusions are not specific to one family of rank correlation. The score distributions for these wallets are heavy-tailed: most wallets have near-zero scores while a handful of hubs carry a large mass. In such situations, Kendall remains well-defined while Pearson correlation on raw scores is dominated by hubs.

- **Family aggregation:** The family summary is simply the average across proxies in that family. For example, the mean τ for the transfer family is simply $\frac{1}{2}(\tau_{\text{in_degree}} + \tau_{\text{in_value}})$. Comparisons between methods on a fixed cohort and proxies are only relative statements about those two methods; they don't imply any universal reputation properties. We rely on confidence intervals for our conclusions, and we don't use any absolute τ cutoff as a decision criterion.
- **Uncertainty:** Sampling uncertainty is estimated using a paired wallet bootstrap ^[34]. We re-sample the matched cohort with replacement $B = 400$ times (seed 42, hard-coded in the configuration), reusing the same index matrix across methods and proxies, so that the resulting resampled statistics are paired across methods. For each proxy τ , for each family mean, and for each of the inter-method differences, we take the 2.5th and 97.5th percentiles of the bootstrap distribution to form a 95% percentile interval ^[35]. Since the population Kendall τ is a U-statistic, the percentile interval is first-order accurate for $n = 5,521$; however, the small number of resamples was chosen to keep the $O(B n \log n)$ computational cost of computing the tie-corrected τ across all methods and proxies tractable, and this limits the precision of the endpoints of the interval to about ± 0.005 .
- **Difference tests:** All comparisons on which our argument rests are pre-specified as paired contrasts, and tested using the same resamples. We compute the bootstrap distribution of $\Delta\tau = \tau_A - \tau_B$ resample by resample and report the 95% percentile interval for it. A contrast is referred to as a difference when the interval does not contain zero. The five pre-specified contrasts between the two single-layer methods are (i) EndorseRank on its allowance family minus EndorseRank on the transfer family; (ii) AWP on the transfer family minus EndorseRank on the transfer family; (iii) EndorseRank on the allowance family minus AWP on the allowance family; (iv) AWP minus EndorseRank on the Sybil-stability family; and (v) En-

dorseRank on `in_approve_degree` minus EndorseRank on `in_degree`. The remaining eight contrasts are between the hybrid operators and the single-layer method that leads each family, respectively, C-PR and S-PR each minus AWP on the transfer family, minus EndorseRank on the allowance family, and minus both on the Sybil-stability family; they include the secondary rule in Section 3.7. Paired resampling cancels out the between-resample covariance in the two coefficient estimates, so the resulting contrast interval will be narrower than the difference between the two separate confidence intervals.

- **Directionality and imputation:** Proxies are ordered such that higher values correspond to better reputations. Wallets that are missing in a particular proxy source get imputed with zeros after being re-aligned to $\mathcal{W}$. This approach is a conservative choice when working with sparse transactions and is implemented consistently across both scoring approaches. Every wallet present in one dataset is also present in the other; the set of wallets included in the alignment process always corresponds to the entire intersection of wallets in the relevant primary table.
- **What is excluded:** There is no causal inference attempted here (reputation does not cause future approvals in the experimental sense), no supervised learning models are trained or evaluated using separate training and testing sets, and no hypothesis tests with corresponding p -values are performed. Rather, we examine whether the confidence interval for the pre-specified contrast excludes the null value. The objective of this study is to determine the degree of alignment between our proposed reputation metric and those established in prior work, and to evaluate the relative computational requirements of each scoring procedure.

3.11 Computational benchmark (Claim C1)

Claim C1 compares the computational performance of EndorseRank to that of AWP. The benchmarking framework evaluates the running time of the eigenvector computation independently from the one-time cost of building the underlying graph representation, since the latter is largely determined by data loading and pre-processing overhead, which would likely be amortized in a produc-

tion scoring service that re-computes reputation scores only periodically, while the former represents the primary cost of executing either algorithm.

- **Timed component:** For each scoring procedure, we construct the weighted adjacency matrix once before entering the timing loop; extraction, decoding, aggregation of allowances and the exponential decay of older transactions are not included in the timing measurement. We measure the execution time of the solver, which forms the sparse matrix from the edge list and then applies power iteration using damping factor $d = 0.85$, convergence threshold $\tau_{\text{tol}} = 10^{-8}$, and maximum number of iterations $I_{\text{max}} = 300$. Both steps in this portion of the measurement are $O(|E|)$ and both steps would need to be repeated at each update of the scoring function in a production deployment. The parameters reported here are identical to those used to compute the alignment scores, which implies that the alignment scores and execution times are based on the same underlying graph.
- **Measurement protocol:** Each configuration is measured in a single session using a consistent measurement protocol. An initial non-timed iteration of the solver is performed to prime internal caches and invoke any lazy initialization routines; this is the only run for which we track peak memory usage (we do not track memory usage during the timed runs since it would significantly increase the elapsed time). We then perform five timed iterations. The statistics reported include the mean and standard deviation of the five run times, the average number of iterations to reach convergence in the timed runs, the maximum memory usage observed during the warm-up run, and the size of the input graph (number of nodes and edges). Measurement results within the session are memoized using a key which is formed from the edge multiset signature, damping, tolerance, iteration limit, and repeat number; when the same graph appears under the same settings (in the main benchmark, the $d = 0.85$ row of the damping sweep, the in-cohort scaling stage with $n = 5,521$, and the expanded pool stage, whose induced subgraph is the cohort graph; Section 3.12.1), the tables thus report the same values by design, not independent measurements of the same entity. I include edge counts in

every runtime table since they are the main structural reason for runtime variation in $O(|E|)$ per-iteration algorithms ^[15]. I include the runtimes themselves in the machine-readable output (Section 3.13).

- **Cohorts:** For the main benchmark, I use the matched wallet cohort ($n = 5,521$). For node-count sensitivity, I use deterministic subsamples of the expanded wallet pool (Section 3.12).
- **Fairness:** Since I compare AWP to EndorseRank, I ensure both use the same solver, tolerance, iteration limit, damping (except where I do a damping sensitivity sweep), and seed wallet set. Differences therefore are due to differences in the graph’s density and weight structure (snapshot of allowances vs. time-decayed transfer history) rather than implementation. I include hardware and software information in my evaluation manifest, so that one may interpret absolute runtimes relative to the experimental machine; I prefer reporting speedup ratios (AWP time divided by EndorseRank time) for comparing methods, and I always include the corresponding edge ratio next to speedup.

3.12 Scaling and robustness extensions

I also investigate the runtime sensitivity to node count on a larger address set, and check whether EndorseRank remains aligned to AWP under parameter and sample perturbations. This supports H3 (scalability to larger n) and allows me to see if my conclusions at the family level extend beyond my chosen sample. These are not alternative primary samples: unless otherwise stated, alignment claims in Chapter 4 apply to the matched cohort, and node-count sensitivity applies to the expanded pool.

3.12.1 Expanded wallet pool and node-count sensitivity of runtime

I define the expanded wallet pool $\mathcal{W}_{\text{pool}}$ to be the union of the matched cohort (5,521 addresses) with 93,559 additional addresses sampled from the monthly Arbitrum One ERC-20 activity in my observation window (I take the supplemental active-wallet parquet created by my sampling script),

giving me $N = |\mathcal{W}_{\text{pool}}| = 99,080$ addresses for this study. I create intermediate-sized benchmarks by deterministic SHA256 subsampling with seed string `benchmark-tier2-v1`: wallets are sorted by

SHA256(seed || address) with seed `benchmark-tier2-v1`

and the first n wallets form the subsample. The seed string is a fixed constant, part of my configuration. The reported stages are $n = 10,000$, $n = 50,000$ and the full pool $n = N = 99,080$, all on the same approval and transfer parquet. In-cohort runs with $n \leq 5,521$ appear in Appendix 8.1. The deterministic hashing eliminates the non-reproducibility of the pseudorandom draws across library versions.

Two features of this approach bound the scope of what the experiment can demonstrate. First, at each stage the graph is the subgraph of the cohort-window approval and transfer parquet induced by the sampled wallets, so $|E|$ at any stage is a function of the wallets selected by the hash order and is not monotone in n by construction. Second, the supplemental pool addresses were intended to have their own approval and transfer logs extracted as a secondary step; that extraction did not finish (Appendix 8.1), so the full-pool stage adds nodes but no additional edges beyond the cohort parquet and reproduces the cohort edge counts precisely. The pool stages therefore capture how runtime reacts to adding isolated or weakly connected nodes at fixed edge counts; they do not scale the edge count that drives per-iteration costs. Per-stage $|V|$ and $|E|$ for both methods are presented in every scaling table. The scaling statement in Chapter 4 is restricted to what those columns imply.

3.12.2 Damping sensitivity

We sweep PageRank damping over $d \in \{0.75, 0.85, 0.95\}$ on the matched cohort, holding the graphs and proxies constant. The default $d = 0.85$ corresponds to the classical web PageRank case^[15] and the AWP baseline^[3]. Smaller d increases teleportation and shrinks scores; larger d places more weight on multi-hop paths and may slow convergence. The sweep probes whether alignment by allowance and transfer families is sensitive to the teleportation probability.

3.12.3 Top-token subgraph

We extract a subgraph containing only the top-20 ERC-20 tokens in terms of combined allowance and transfer volume during the window. We recompute EndorseRank and AWP on that subgraph to evaluate sensitivity to long-tail tokens and potential spam. If family-level results flip when low-volume tokens are excluded, then the main finding would be fragile to the set of tokens considered; if results remain stable, it supports interpreting the full-token graphs as a proxy for major economic transactions.

3.12.4 Sample-definition sensitivity

We recalculate alignment statistics on subsamples drawn from the expanded wallet pool at each staging point with the same deterministic seed. The main sample for each alignment claim is the matched wallet cohort ($n = 5,521$), with its estimated coefficients and confidence intervals. The pool subsamples differ from the cohort in both size and composition: they include wallets that appear in only one of the two graphs, and whose score and proxy in the other graph are zero even after alignment; such ties at zero inflate rank agreement mechanically. We report coefficients at each stage as a sensitivity check on the sample, not as evidence of stability; the question is whether EndorseRank and AWP maintain their relative ordering on the transfer and allowance families when the sample definition changes.

3.13 Reproducibility

Every empirical table in Chapter 4 was generated by an open and versioned evaluation pipeline, not manually transcribed. Once the Phase 1 extracts are written to disk, a single evaluation driver recomputes all construct checks, all alignment statistics with bootstrap confidence intervals, and all runtime benchmarks, then outputs a single machine-readable summary; another stage exports Chapter 4 and Appendix 8.1 tables from that file. The evaluation driver can run either over cached ledger extracts or over synthetic fixtures; it can also toggle the damping, top-token, and sample-definition

sweeps. The observation window, the damping value, the decay parameters, the event-topic IDs, the bootstrap resample count and the bootstrap seed, and the benchmark seeds are all stored in one versioned configuration file. In fixture mode, the pipeline runs entirely offline, without any cloud credentials, so the implementation can be checked locally.

Reproducible features include: deterministic wallet subsampling using SHA256 hashes; fixed PageRank tolerance and maximum number of iterations; a single bootstrap seed and a single resample-index matrix; the proxy formulas specified in Section 3.9; the benchmarking procedure of Section 3.11, with the per-run timing data kept in the logs; the coupling weight λ and the holdout logic of Section 3.7, which were fixed in the configuration file before the run started; and the extracted manifest files. The tables reporting alignment and benchmark figures can be rebuilt from those same extracts without issuing new BigQuery queries. The machine-readable summaries from which all tables were derived, along with the active configuration, are committed to the Git repository under `results/`, directly beside the source code, so the numbers in this paper can be cross-checked against the summaries without rerunning the pipeline. The GitHub repository URL, the exact commit hash used to generate the tables in this paper, the configuration file, and the summary files are listed in Appendix 8.1.

3.14 Sampling considerations and missingness

The matched cohort is a purposefully constructed sample, not a simple random draw of all Arbitrum addresses. An address must exhibit nonzero degree in the approval or transfer reputation subgraphs to be eligible, so the findings apply only to wallets that show some kind of endorsement- or transfer-graph footprint. Wallets that are dormant, that act solely as long-term holders, or that only transact via non-ERC-20 channels are excluded by design.

Missing data comes in a few forms. Tokens whose decimal precision is unknown might be pruned or set to zero during the preprocessing stage; any such choice is logged in the pipeline. Approvals that do not emit standard ERC-20 log events (for example, approvals on non-standard tokens) are

completely opaque to EndorseRank. Transfers that take place off-chain or inside custodial accounts are simply not ingested into AWP. Both methods are therefore biased toward on-chain, standard-token behavior. This is fine for an on-chain reputation study, but it matters when extrapolating to the whole set of users in a particular protocol.

Additional addresses included in the broader wallet universe ($N = 99,080$) improve node coverage for the runtime experiments. The alignment claim is therefore always calculated with respect to the matched cohort ($n = 5,521$); the node-count sensitivity analysis instead uses the pool.

3.15 Implementation notes

PageRank is implemented in Python using a SciPy sparse transition operator and NumPy power iteration. Edge lists are constructed once for each configuration; the time benchmarks exclude I/O and preprocessing, covering only operator assembly and iteration (Section 3.11). Randomness is present solely through optional fixture generation; evaluation against real data is deterministic given the parquet inputs and configuration seeds. Minor floating-point differences between platforms are expected, and would be negligible compared to the three-decimal-place precision of the τ metric. Figures are produced from the same evaluation artifacts as the tables, ensuring consistency between the reported numerical values and the visualizations.

3.16 Ethics

This research makes use of only publicly available blockchain data; there are no human subjects. The institutional ethics review process (Unisa SoC/CAES) was completed before any empirical data collection. The ethics clearance certificate (reference [Unisa SoC/CAES ethics clearance ref: TBD], dated [TBD]) is included in Appendix 8.6.1.

- Privacy and anonymity: Addresses on Arbitrum One are pseudonyms that do not inherently reveal their real-world identity ^[2]. While pseudonymity is not the same as anonymity, it is theoretically possible for a determined adversary to associate an address with a person via

exchange deposits, online profiles or heuristics. In this work we explicitly choose not to make such links. Our data collection focuses solely on on-chain Approval and Transfer events; no scraping of off-chain personally identifiable information (such as IP addresses, exchange KYC records, social media handles, or deanonymization graphs) was performed. We did not cluster addresses into real-world entities; nor did we publish leaderboards of addresses likely to be personally identifiable; nor did we attempt to combine a wallet’s transaction history with other databases to infer identity. The published results consist only of aggregate measures (correlation coefficients, runtimes, family means) and not individualized rankings of named participants. Examples quoted in the main text refer to broad categories of user (for example, ‘spenders with high allowances’) and do not mention specific addresses.

- **Data minimization:** Processed data tables contain only the fields needed for building the graph and conducting the checks (i.e., the relevant addresses, token amounts and timestamps). The raw logs are retained to allow the research pipeline to be audited, but will not be released for the purpose of deanonymizing users. Query filters have been scoped to wallets wherever possible, to avoid running indiscriminate full-chain dumps outside the scope of the study design.
- **Data usage and compliance:** All queries ran against public datasets hosted within the `bigquery-public-data.goog_blockchain_arbitrum_one_us` dataset, and followed the Google Cloud terms of service and rate limits. Dry-run byte estimation and per-query maximums prevented unnecessary scans. No transactions were sent, no smart contracts were invoked in a way that might interfere with normal operation of the network, and no privileged or non-public indexing endpoints were accessed.
- **Fairness and transparency:** Graph-based reputation systems may undervalue new participants, low-frequency actors, or those wallets that do not grant public approvals [2]. We therefore examined both EndorseRank and AWP for these kinds of structural biases: in the allowance graph, for example, an address that has received no incoming spend authority is at a disadvantage relative to those receiving spend authority; transfer graphs prefer addresses embedded

in value-flow networks. Limitations, including cold-start effects and Sybil susceptibility of approval graphs, are discussed in Chapter 5. All code paths, parameter choices, and evaluation metrics are documented in the open pipeline (Appendix 8.1) to enable peer review and independent replication.

- **Responsible disclosure and impact:** Although EndorseRank may inform risk assessment in DeFi contexts, it is not financial advice, credit advice, or a consumer reporting product. Human oversight remains necessary for lending, trading, and compliance decisions. Potential adversarial manipulation (for example, Sybil attacks on G_{approve} or wash transfers on G_{transfer}) is acknowledged and discussed in Chapter 5. The study reports aggregated cohort-level findings rather than wallet-level rankings intended for targeting individual users. Any future operational deployment would require additional governance, appeal mechanisms, and fairness monitoring beyond the scope of this research.
- **Summary of methodological commitments:** The design fixes the chain, window, cohort rules, graph formulas, proxy definitions, bootstrap protocol, timing protocol, and statistical tests before interpreting results. Primary claims compare only EndorseRank and AWP on the matched wallet cohort and the spender holdout; the expanded pool and sensitivity sweeps test efficiency and the dependence of results on parameters and sample definition.

Chapter 4 reports the empirical outcomes of this protocol as an Arbitrum One case study: it first characterizes the dataset and cohort, then presents the runtime benchmark and its node-count sensitivity, the sensitivity sweeps, the alignment tables with intervals and contrasts, the rank-divergence test, the same-window position of the coupled operator, and the two-label temporal holdout, and closes by stating Claims C1–C5.